



Trackam

Track your coding grind.

Comprehensive project handoff — an affordable, self-hostable, WakaTime-compatible coding time tracker built for African developers.

Node.js + TypeScript

PostgreSQL

Open Source

Repository: github.com/OutstandingVick/trackam

Document: Engineering & product handoff · **Version:** 0.1.0 · **Date:** 20 July 2026

Status: MVP complete — core ingestion, dashboard & deploy ready

Contents

1. Executive summary
2. Problem & market
3. How it works (the core idea)
4. Architecture overview
5. Technology stack
6. Data model
7. API reference
8. The coding-time algorithm
9. Authentication & security
10. Frontend & design system
11. Configuration (environment variables)
12. Local development
13. Deployment
14. Repository & project structure
15. Testing & verification status
16. Known limitations
17. Roadmap & monetization
18. Onboarding checklist for a new developer

1 · Executive summary

Trackam is an automatic coding-time tracker — like WakaTime — that is **free to self-host** and **100% compatible with the official WakaTime editor plugins**. It targets developers for whom WakaTime's US\$9–14/month subscription is unaffordable, starting with the Nigerian / African developer community.

Because the WakaTime editor plugins are open source and speak a documented HTTP API, Trackam does not ship its own editor extensions. Instead it re-implements the WakaTime ingestion API. Developers keep the plugin they already use (VS Code, Cursor, JetBrains, Vim, etc.) and simply point its `api_url` at their Trackam server.

What is built today (MVP)

- WakaTime-compatible heartbeat ingestion (bulk & single) and editor status-bar endpoint.
- Email/password accounts with per-user API keys.
- Dashboard: Today / 7-day / 30-day totals, daily chart, and breakdowns by project, language, editor and OS.
- A distinctive neo-brutalist UI in the brand palette.
- One-command Docker Compose deployment (app + Postgres + migrations).

2 · Problem & market

WakaTime is the de-facto coding-time tracker, but its pricing puts it out of reach for many developers in emerging markets. The paid tiers are **\$9/mo (Basic)** and **\$14/mo (Premium)**; crucially, the free tier limits dashboard history to roughly the last week, so a developer's own historical data is effectively paywalled.

Why the cost feels high

- The heavy lifting (editor plugins + `wakatime-cli`) is open source and runs on the user's machine — the subscription mostly pays for hosted dashboards and **history retention**.
- Access to your own past data is the primary thing being sold.
- Priced in USD, the amount is disproportionately large in NGN terms.

Trackam's wedge

Give developers unlimited history and the same automatic tracking for free by self-hosting, with an optional low-cost hosted tier priced in naira later. The addressable audience includes bootcamps, student developer communities, and indie devs across Africa.

3 · How it works (the core idea)

WakaTime editor plugins emit **heartbeats** — small events describing what you are editing right now (file, project, language, branch, whether you're typing, etc.). The plugin batches these and POSTs them to whatever `api_url` is configured. Trackam implements the same endpoints those plugins expect, so it is a drop-in server.

```
Editor (WakaTime plugin)
  | batches heartbeats
  ▼
POST /api/v1/users/current/heartbeats.bulk — Authorization: Basic base64(api_key)
  |
  ▼
Trackam API (Express) → PostgreSQL (heartbeats table)
  |
  ▼
Dashboard → GET /api/stats?range=... → duration algorithm → charts
```

The only thing a developer changes is two lines in `~/.wakatime.cfg`: `api_url` and `api_key`.
No new software to install.

4 · Architecture overview

Trackam is a single Node.js service that serves three concerns from one process:

- **WakaTime-compatible ingestion API** (mounted at `/api/v1`) — authenticated by API key.
- **Account & stats API** (`/api/account`, `/api/stats`) — authenticated by a signed session cookie.
- **Static frontend** — the dashboard SPA served from `public/`.

State lives entirely in PostgreSQL. There is no background worker or queue; coding-time durations are computed on-read from raw heartbeats, which keeps ingestion cheap and the system easy to reason about at MVP scale.

RUNTIME

Single stateless Node process (horizontally scalable behind a load balancer).

STORAGE

PostgreSQL — raw heartbeats + users.

COMPUTE MODEL

Durations derived at query time (no pre-aggregation yet).

DEPLOY UNIT

Docker image; Compose bundles app + DB + migration job.

5 · Technology stack

Layer	Choice	Notes
Language	TypeScript (ES2022, NodeNext)	Strict mode enabled.
Runtime	Node.js ≥ 18	ESM modules.
Web framework	Express 4	JSON body limit 2 MB.
Database	PostgreSQL (via <code>pg</code>)	Custom type parsers for INT8/FLOAT8.
Auth	bcryptjs, jsonwebtoken, cookie-parser	Password hashing + JWT session cookie.
Config	dotenv	<code>.env</code> driven.
Frontend	Static HTML + vanilla JS + Tailwind (Play CDN)	No build step for the UI today.
Dev tooling	tsx (watch), tsc (build)	Smoke test runs on compiled output.
Packaging	Docker (multi-stage, node:20-alpine) + Compose	Postgres 16-alpine.

NPM scripts

Script	Command	Purpose
<code>dev</code>	<code>tsx watch src/index.ts</code>	Hot-reload dev server.
<code>build</code>	<code>tsc -p tsconfig.json</code>	Compile to <code>dist/</code> .
<code>start</code>	<code>node dist/index.js</code>	Run compiled server.
<code>migrate</code>	<code>tsx src/migrate.ts</code>	Apply <code>db/schema.sql</code> .
<code>test</code>	<code>tsc && node dist/smoke.js</code>	Core-logic smoke checks.

6 · Data model

Two tables. The schema lives in `db/schema.sql` and is applied by the migrate script.

users

Column	Type	Notes
id	UUID (PK)	Default <code>gen_random_uuid()</code> .
email	TEXT, UNIQUE	Stored lower-cased.
username	TEXT, UNIQUE	3–32 chars: letters, numbers, <code>_</code> or <code>-</code> .
password_hash	TEXT	bcrypt (cost 10).
api_key	UUID, UNIQUE	Used by editor plugins; rotatable.
created_at	TIMESTAMPTZ	Default <code>now()</code> .

heartbeats

Column	Type	Notes
id	BIGSERIAL (PK)	
user_id	UUID (FK → users)	ON DELETE CASCADE.
time	DOUBLE PRECISION	Epoch seconds (fractional) from the CLI.
entity, type, category	TEXT	File path, "file", "coding" etc.
project, branch, language	TEXT	Editing context.
dependencies	TEXT	Comma-joined.
lines, lineno, cursorpos	INTEGER	Position metadata.
is_write	BOOLEAN	Save vs read.
editor, operating_system, machine	TEXT	Parsed from User-Agent when absent.
created_at	TIMESTAMPTZ	Server receive time.

Indexes

- `idx_heartbeats_user_time (user_id, time)` — fast windowed reads.
- `uniq_heartbeat (user_id, time, entity, type)` — dedupes CLI retries (INSERT ... ON CONFLICT DO NOTHING).

7 · API reference

WakaTime-compatible (mounted at `/api/v1`)

Authenticated by API key. `api_url` in the plugin config must therefore end in `/api/v1`.

Method	Path	Purpose
POST	<code>/users/current/heartbeats.bulk</code>	Receive a batch of heartbeats.
POST	<code>/users/current/heartbeats</code>	Receive a single heartbeat or array.
POST	<code>/users/:user/heartbeats[.bulk]</code>	Same, alternate user-scoped form.
GET	<code>/users/current/statusbar/today</code>	Today's total for the editor status bar.

Account (`/api/account`)

Method	Path	Auth	Purpose
POST	<code>/signup</code>	—	Create account, set session, return API key.
POST	<code>/login</code>	—	Email or username + password.
POST	<code>/logout</code>	cookie	Clear session.
GET	<code>/me</code>	cookie	Current user + API key.
POST	<code>/regenerate-key</code>	cookie	Rotate the API key.

Stats (`/api`)

Method	Path	Auth	Purpose
GET	<code>/stats?range=today week month</code>	cookie	Totals, daily series & breakdowns.
GET	<code>/healthz</code>	—	Liveness + DB check.

Auth header (from wakatime-cli)

```
Authorization: Basic base64(api_key)
```

Trackam also accepts `Bearer <key>` and an `?api_key=` query param, and strips a trailing colon from the decoded Basic value.

8 · The coding-time algorithm

Coding time is derived from raw heartbeats the way WakaTime does it. Heartbeats are sorted by time; the gap between each consecutive pair is added to the total **only if** it is below a configurable timeout (`HEARTBEAT_TIMEOUT_SECONDS` , default **900s / 15 min**). Larger gaps are treated as a break between sessions, so idle time is not counted. Each interval is attributed to the project / language / editor / OS of the heartbeat that *starts* it.

```
for i in 1..n:
    delta = t[i] - t[i-1]
    if 0 < delta < timeout:
        total += delta
        attribute delta to project/language/editor/os of heartbeat[i-1]
```

The implementation lives in `src/utls/duration.ts` . Per-day series are produced by bucketing heartbeats into local-day groups and running the same routine per day. Editor and OS are inferred from the `User-Agent` string when the plugin doesn't send them explicitly (`src/utls/useragent.ts`).

This on-read approach is simple and accurate for MVP volumes. At larger scale, add a nightly pre-aggregation (daily summaries table) to keep dashboard queries fast.

9 · Authentication & security

Two auth paths

- **Dashboard:** bcrypt-hashed passwords; on login a JWT is signed with `JWT_SECRET` and stored in an `httpOnly` cookie (`trackam_session` , 30-day expiry, `secure` in production, `SameSite=Lax`).
- **Ingestion:** per-user UUID API key sent via HTTP Basic, validated for UUID shape before a DB lookup.

Current safeguards

- Parameterized SQL everywhere (no string interpolation).
- HTML escaping in the dashboard render path.
- Input validation on signup (email/username/password rules).
- Unique constraints surface friendly 409s instead of leaking DB errors.

Hardening TODO before scale / public hosting: set a strong `JWT_SECRET` (the app warns if the insecure default is used); add rate limiting on auth + ingestion; consider hashing API keys at rest; add CSRF protection if any cookie-authed state-changing endpoints are exposed cross-site; put the service behind TLS (reverse proxy).

10 · Frontend & design system

The dashboard is a single static page (`public/index.html` + `public/app.js`) that talks to the JSON API. The visual language is intentionally **neo-brutalist and playful** — thick black borders, hard offset shadows, chunky rounded cards, animated background blobs, rainbow data bars, and Naija-flavoured microcopy — so it does not feel like a generic SaaS dashboard.

Brand palette

Colour	Hex	Usage
■ Near black	#0A0A0A	Text, borders, shadows.
■ Vivid orange	#E8642F	Accent / range state.
■ Hot pink	#FF1097	Primary CTA / logo.
■ Soft pink	#FECDEC	Card fill.
■ Lime green	#9CE404	Success / stat card.
■ Pale yellow	#FEFFB2	Page canvas.
■ Pale lavender	#EED6FF	Card fill.
■ Light lavender	#ECCEFF	Inputs.
■ Electric violet	#9800FF	Secondary CTA / links.
■ Deep purple	#5900C0	Accents.
■ Light grey	#E0E0E0	Tracks / dividers.

Type: Bricolage Grotesque (display) + Space Grotesk (body). **Motion:** springy bar growth, press/translate on buttons, hover card-pop, floating blobs, logo wiggle.

Tailwind is loaded via the Play CDN for MVP speed. Before production, switch to a compiled Tailwind build (smaller, faster, no runtime CDN dependency).

11 · Configuration (environment variables)

Variable	Default	Description
PORT	3000	HTTP port.
NODE_ENV	development	Toggles secure cookies + warnings.
BASE_URL	http://localhost:3000	Public URL (no trailing slash).
DATABASE_URL	postgres://trackam:trackam@localhost:5432/trackam	Postgres connection string.
JWT_SECRET	— (required)	Signs dashboard session cookies.
HEARTBEAT_TIMEOUT_SECONDS	900	Max gap counted as continuous coding.

A template is provided in `.env.example` ; copy it to `.env` .

12 · Local development

```
# Requires Node 18+ and a running PostgreSQL
npm install
cp .env.example .env          # set DATABASE_URL + JWT_SECRET
npm run migrate               # create tables
npm run dev                   # http://localhost:3000 (hot reload)
```

Then sign up in the dashboard and copy the generated `~/wakatime.cfg` snippet into your editor's WakaTime config.

13 · Deployment

Designed for a single cheap VPS ($\approx$ 1 vCPU / 1 GB RAM). Docker Compose starts three services: `db` (Postgres), a one-shot `migrate` job, and the `app`.

```
cp .env.example .env          # set a strong JWT_SECRET + BASE_URL
docker compose up -d --build
# dashboard on http://your-host:3000
```

Recommended production additions

- Reverse proxy (Caddy/Nginx) for TLS + gzip; Caddy gives automatic HTTPS.
- Automated Postgres backups (e.g. nightly `pg_dump` to object storage).
- Uptime monitoring against `/healthz`.

14 · Repository & project structure

Remote: github.com/OutstandingVick/trackam · branch `main` . Notable commits: initial MVP, then a neo-brutalist frontend redesign.

```
src/
  index.ts           # Express app + wiring
  config.ts          # Env config
  db.ts              # Postgres pool + type parsers
  migrate.ts         # Runs db/schema.sql
  auth.ts            # Password/session + API-key auth
  smoke.ts           # Core-logic tests
  routes/
    wakatime.ts      # WakaTime-compatible ingestion + statusbar
    account.ts       # Signup / login / API key
    stats.ts         # Dashboard stats API
  utils/
    duration.ts      # Heartbeat → coding-time algorithm
    useragent.ts     # Parse editor/OS from User-Agent
  public/            # Dashboard (index.html + app.js)
  db/schema.sql      # Database schema
  Dockerfile · docker-compose.yml · .env.example · README.md
```

15 · Testing & verification status

Area	Status	Notes
TypeScript build	✅ Passing	<code>npm run build</code> compiles cleanly.
Core logic (duration + UA parse)	✅ Passing	Smoke checks: continuous session, idle-gap split, multi-project attribution, UA parsing.
Frontend render	✅ Verified	Auth + dashboard rendered live in a browser.
Full DB-backed HTTP flow	⚠️ Not yet run	No Postgres available in the build sandbox; run locally via Docker Compose to verify ingestion end-to-end.
Automated integration tests	✗ TODO	Add supertest-based API tests + a seeded Postgres in CI.

16 · Known limitations

- Durations are computed on every request; no pre-aggregation yet (fine at MVP scale).
- Tailwind via CDN, not a compiled build.
- No rate limiting on auth or ingestion endpoints.
- API keys stored in plaintext (UUIDs) rather than hashed.
- No email verification / password reset flow.
- Day bucketing uses server local time, not per-user timezone.
- No teams, leaderboards, badges, or payments yet (see roadmap).

17 · Roadmap & monetization

Near-term features

- **Private leaderboards** — high value for Nigerian dev communities & bootcamps.
- **GitHub README time badges** — free organic marketing.
- **Weekly email summaries.**
- Per-user timezones and goals.

Monetization (deferred)

Core stays free/open-source. An optional hosted tier priced in naira via **Paystack** (cards, bank transfer, USSD) — e.g. ₦1,000–2,000/month — can fund free hosting for those who self-host-averse, without paywalling history.

Scale hardening

- Daily summary aggregation table + background job.
- Rate limiting, API-key hashing, CI integration tests.
- Compiled Tailwind build; optional multi-region.

18 · Onboarding checklist for a new developer

1. Clone `github.com/OutstandingVick/trackam`.
2. `npm install`; copy `.env.example` → `.env` and set `JWT_SECRET` + `DATABASE_URL`.
3. Start Postgres (local or `docker compose up db`), then `npm run migrate`.
4. `npm run dev`; open the dashboard and sign up.
5. Point a real editor's WakaTime plugin at `http://localhost:3000/api/v1` using your API key, code for a minute, and confirm heartbeats appear.
6. Read `src/routes/waketime.ts` (ingestion) and `src/utls/duration.ts` (the algorithm) first — they are the heart of the system.
7. Run `npm test` before pushing.

Trackam · self-hosted, WakaTime-compatible coding time tracking · built for African developers · this handoff generated 20 July 2026.